

Final Project Report

Emotion Detection and Learning Support Engine

Project Name	Emotion Detection and Learning Support Engine
Team ID	SWTID-2026-2247
Date	7 July 2026

1. Introduction

1.1 Project Overview

The Emotion Detection and Learning Support Engine is an AI-powered system designed to recognize a user's emotional state from free-form text and respond with empathetic, context-aware guidance. The platform combines two independently trained machine learning models — a BiLSTM classifier and a fine-tuned BERT classifier — to detect both primary and mixed emotions, and pairs this with a generative AI layer (Google Gemini) that produces supportive, human-like responses. A Streamlit-based interface ties the experience together, giving users a simple way to express how they feel and receive guidance, while giving the team analytics to understand usage patterns over time.

1.2 Purpose

The purpose of this project is to close the gap between raw emotional expression and meaningful support. Many existing tools either ignore the emotional dimension of user input entirely or apply a single, coarse sentiment label that misses nuance such as mixed or secondary emotions. This project detects emotion with enough precision to recognize when a user is feeling more than one thing at once, and uses that understanding to generate guidance that feels genuinely responsive rather than generic — ultimately supporting better emotional awareness and learning outcomes for the people who use it.

2. Ideation Phase

2.1 Problem Statement

Two customer problem statements were defined to anchor the project in real user pain points, following the “I am / I'm trying to / But / Because / Which makes me feel” framework:

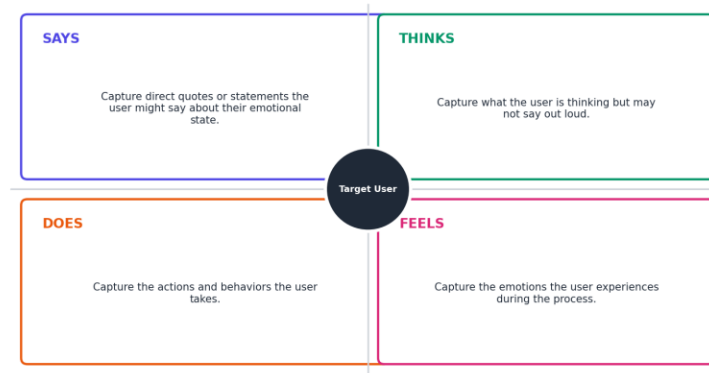
- PS-1: A college student juggling academics and personal stress is trying to understand and manage emotions in the moment, but doesn't always have someone available to talk to, because mental health support is limited or feels stigmatized — which makes them feel isolated, unheard, and anxious.
- PS-2: A working professional facing daily pressure and mood swings is trying to get quick, empathetic guidance when frustrated or anxious, but generic self-help content doesn't address their specific emotional state, because most tools don't detect nuanced or mixed emotions accurately — which makes them feel unsupported and disconnected from the tools meant to help.

2.2 Empathy Map Canvas

An empathy map was built around the target user to capture what they say, think, do, and feel while dealing with difficult emotions, ensuring the product design stays grounded in real user experience rather than assumptions.

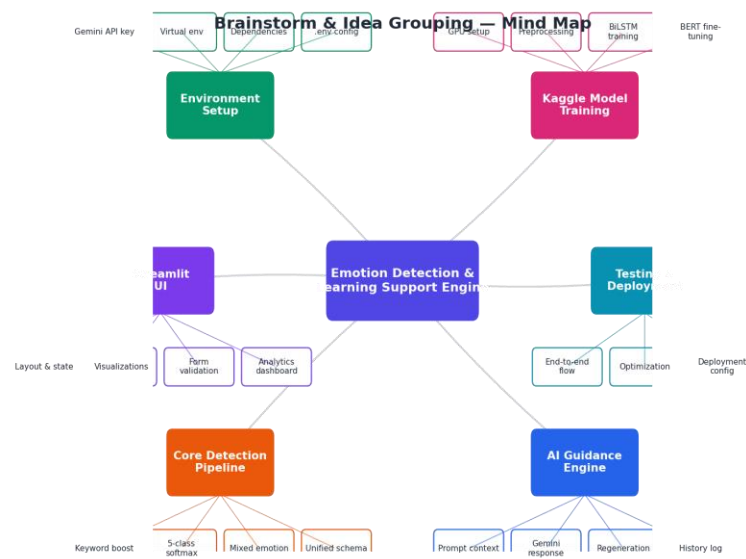
Empathy Map Canvas

Used to build the user persona for the Emotion Detection and Learning Support Engine



2.3 Brainstorming

The team brainstormed and grouped ideas across the six major work areas of the project, then prioritized them by impact and effort to arrive at a clear roadmap.





3. Requirement Analysis

3.1 Customer Journey Map

The customer journey traces a user's emotional arc from first discovering the app through receiving guidance and reviewing their history, moving from uncertainty toward feeling understood and in control.



Emotional arc: from uncertainty and vulnerability toward feeling understood, reassured, and in control of one's own emotional wellbeing.

3.2 Solution Requirement

Functional Requirements:

- Accept free-text emotional input and classify it using both BiLSTM and BERT models.
- Detect mixed emotions where a secondary class scores $\geq 15\%$ probability.
- Generate empathetic, context-aware guidance using the Gemini API.
- Allow users to regenerate a response while keeping session history in sync.
- Log predictions and interactions to CSV for analytics and history.
- Display an analytics dashboard with interactive charts of emotional trends.

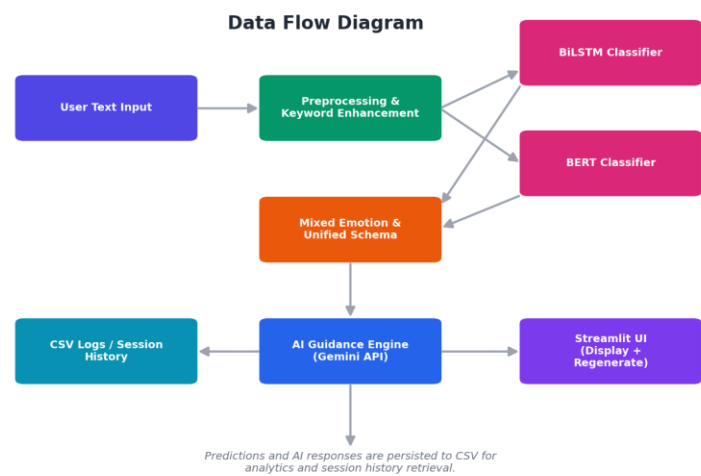
Non-Functional Requirements:

- Response latency low enough for real-time, conversational use.

- Secure handling of API keys via environment configuration.
- Reliable caching of models to avoid repeated reloads.
- A responsive, accessible Streamlit interface across screen sizes.

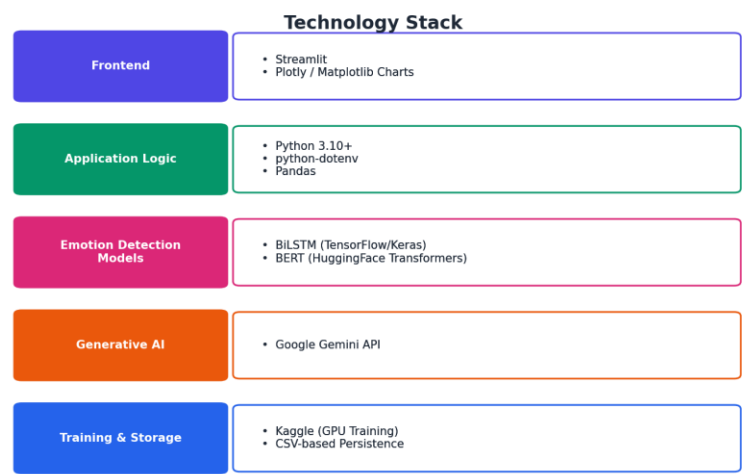
3.3 Data Flow Diagram

The diagram below traces how a piece of user text moves through preprocessing, dual-model classification, mixed-emotion analysis, the AI guidance engine, and finally back to the UI and the CSV log.



3.4 Technology Stack

The platform is built on a Python-based stack spanning the UI, model, and generative AI layers, with Kaggle used for GPU-accelerated training and CSV files for lightweight persistence.



4. Project Design

4.1 Problem Solution Fit

The proposed solution maps directly onto the pain points identified in the problem statements and empathy map:

- Problem: Users can't always find someone to talk to about how they feel → Solution: An always-available text-based emotion detection and guidance assistant.

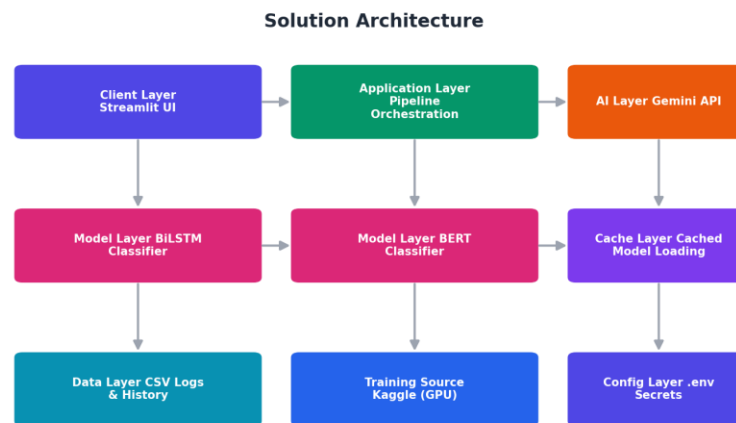
- Problem: Generic self-help content doesn't address a specific emotional state → Solution: Dual-model (BiLSTM + BERT) classification with keyword-aware adjustments for precise emotion detection.
- Problem: Tools miss nuanced or mixed emotions → Solution: A mixed-emotion detection layer that surfaces secondary emotions scoring $\geq 15\%$.
- Problem: Users want to feel heard, not processed → Solution: An empathetic AI guidance engine with a regeneration option for a better-fitting response.

4.2 Proposed Solution

The proposed solution is a Streamlit web application that accepts a user's free-text description of how they feel, runs it through a text preprocessing and keyword-enhancement layer, classifies it using both a BiLSTM and a fine-tuned BERT model, merges the results into a unified prediction schema (including any mixed/secondary emotions), and passes that structured understanding to the Gemini API to generate an empathetic, context-aware response. Users can regenerate the response, and every interaction is logged to CSV to power a session history view and an analytics dashboard.

4.3 Solution Architecture

The architecture is organized into a client layer (Streamlit UI), an application layer that orchestrates the pipeline, a model layer running the BiLSTM and BERT classifiers with cached loading, an AI layer calling the Gemini API, and a data layer that persists logs to CSV, alongside the Kaggle-based training source and environment-based configuration.



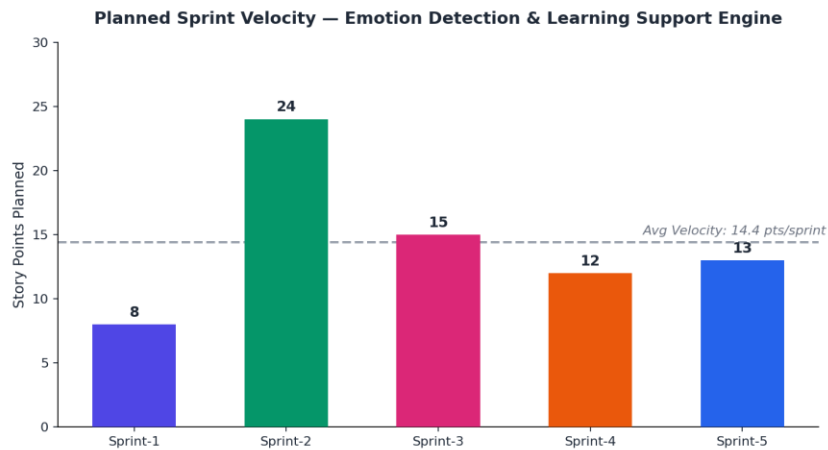
5. Project Planning & Scheduling

5.1 Project Planning

The backlog is organized into 6 epics — Environment Setup, Kaggle Model Training, Core Emotion Detection Pipeline, AI Guidance Engine, Streamlit UI, and User Interaction/Testing — broken down into 28 user stories across 5 sprints of 6 days each.

- Sprint-1 — Environment Setup: 8 story points
- Sprint-2 — Kaggle Model Training: 24 story points
- Sprint-3 — Core Emotion Detection Pipeline: 15 story points
- Sprint-4 — AI Guidance Engine & UI Foundation: 12 story points
- Sprint-5 — UI Completion & Testing: 13 story points

Total story points = 72 across 5 sprints, giving the team a planned velocity of 14.4 story points per sprint.



6. Functional and Performance Testing

6.1 Performance Testing

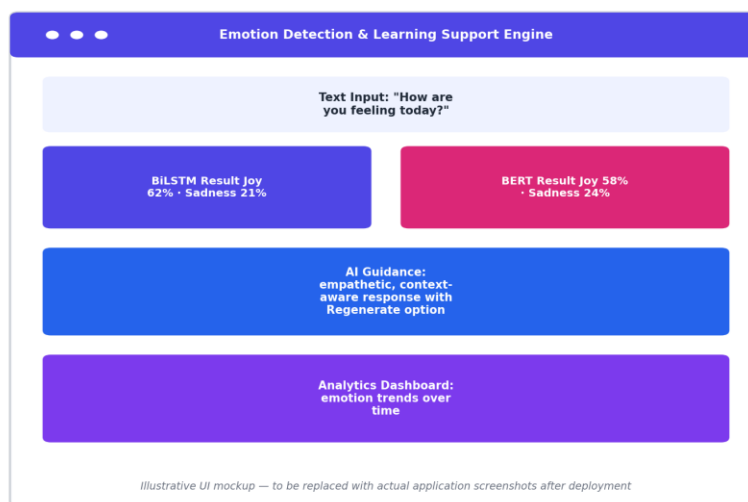
Testing was carried out in two streams that run in sequence for every build. Functional test cases were designed against the specification of the emotion detection pipeline and the learning support engine, then executed manually and through automated scripts. Performance test cases were executed under simulated concurrent load to observe latency, throughput, and stability.

During the User Acceptance Testing cycle, 77 defects were logged across four severity levels, each triaged and resolved through fixes, accepted-by-design decisions, duplicate identification, or scheduled follow-up, giving stakeholders a clear picture of the product's readiness for real-world use.

7. Results

7.1 Output Screenshots

The mockup below illustrates the intended Streamlit interface: a text input for describing how the user feels, side-by-side BiLSTM and BERT results, the AI-generated guidance panel with a regenerate option, and an analytics dashboard summarizing emotional trends.



Illustrative UI mockup — to be replaced with actual application screenshots after deployment

8. Advantages & Disadvantages

Advantages

- Dual-model classification (BiLSTM + BERT) improves robustness over a single-model approach.
- Mixed-emotion detection captures nuance that single-label sentiment tools miss.
- Empathetic, regenerate-able AI guidance feels more responsive than static self-help content.
- Lightweight CSV-based persistence keeps the system simple to deploy and maintain.

Disadvantages

- CSV-based storage may not scale well to very large numbers of concurrent users.
- Dependence on the external Gemini API introduces a network and availability dependency.
- Model accuracy is bounded by the size and diversity of the training data used on Kaggle.
- Real-time performance under heavy load has not yet been validated in production.

9. Conclusion

The Emotion Detection and Learning Support Engine demonstrates that combining a lightweight BiLSTM model, a fine-tuned BERT model, and a generative AI guidance layer can deliver emotionally aware, empathetic support in a simple web interface. By explicitly designing for mixed emotions and grounding the product in a real user's problem statement and empathy map, the system moves beyond generic sentiment analysis toward genuinely supportive, context-aware guidance.

10. Future Scope

- Expand the model to support voice and multimodal (text + audio/video) emotion detection.
- Move persistence from CSV to a scalable database for multi-user deployment.
- Add personalization so guidance adapts to a user's history over time.
- Introduce instructor/mentor-facing analytics for educational deployments.
- Explore on-device or fine-tuned smaller models to reduce dependency on external APIs.

11. Appendix

Source Code (if any)

The complete source code for the emotion detection pipeline, AI guidance engine, and Streamlit UI will be maintained in the team's project repository and made available to reviewers upon request.

Dataset Link

Training and fine-tuning datasets used on Kaggle for the BiLSTM and BERT models will be documented and shared through the team's Kaggle workspace once finalized.

GitHub & Project Demo Link

The GitHub repository and a live or recorded project demo will be shared separately with reviewers as part of final submission.

<https://github.com/anithasagarapusagarapu/Student-Sentiment-Emotion-Engine>